PixMath: A Bidirectional Architecture for Mathematical Image Synthesis and Symbolic Extraction

1. Mathematical & AI Foundations

The theoretical and computational foundation of the PixMath platform rests on establishing a robust, bidirectional mapping between continuous mathematical domains and discrete pixel matrices. This architecture necessitates two distinct but complementary systems: a Forward Engine capable of rendering complex, closed-form mathematical expressions into high-resolution discrete images, and a Reverse Engine that utilizes advanced artificial intelligence paradigms to extract parsimonious, human-readable algebraic expressions from arbitrary digital image inputs.

Forward Rendering: Continuous Mathematics to Discrete Pixel Matrices

The Forward Engine operates by evaluating continuous two-dimensional coordinate spaces, mapping a given (x, y) coordinate plane directly to RGB pixel vectors. Rather than relying on discrete raster graphics operations or traditional polygon rasterization, the engine constructs images entirely through pure algebraic, trigonometric, and exponential compositions. This approach draws heavy inspiration from parametric mathematical art paradigms, wherein complex, real-world geometries—such as biological structures, fractal tessellations, and intricate organic shapes—are synthesized entirely through layered functional architectures[1].

To achieve this, the system evaluates each pixel coordinate by passing it through a composite function $f(x, y) \to \mathbb{R}^3$. The visual output is dictated by layering several foundational mathematical primitives. Complex figures are routinely generated using highly nested, periodic trigonometric functions. For instance, the parametric mapping of a closed curve resembling a bird in flight or a specialized botanical structure relies on the delicate combination of varying frequencies, phase shifts, and amplitude modulations[1]. The geometry is often defined by manipulating the Cartesian coordinates via functions that exploit the periodic nature of sine and cosine to create repeating textures such as feathers, scales, or ribbed leaves[1].

A specific example of this mathematical synthesis can be observed in parametric equations where an image is defined across tens of thousands of line segments or points. The structural foundation relies on equations such as $(x - A(k))^2 + (y - B(k))^2 = R(k)^2$, where the center coordinates $A(k)$ and $B(k)$, as well as the radius $R(k)$, are defined by deeply nested trigonometric polynomials[4]. For example, the x-coordinate shift $A(k)$ might be parameterized as an expansion of $\sin\left(\frac{\pi}{5}(k/c_1)^7\right)$ modulated by high-order cosines $\cos^{16}(c_2 k)$ to induce rapid, localized oscillations that visually manifest as textural details[4].

These techniques, pioneered in mathematical art, demonstrate how scaling variables and cyclical permutations can deform standard tori or elliptical boundaries into recognizable, complex objects without the use of traditional brushstrokes or vector paths[4].

To bound regions and create sharp, anti-aliased edges, the architecture heavily employs Signed Distance Functions (SDFs). An SDF evaluates the orthogonal distance from any given pixel coordinate (x, y) to a mathematical boundary, returning a positive value if the pixel is outside the boundary, a negative value if inside, and zero exactly on the surface. By combining basic SDFs—such as $f(x, y) = \sqrt{x^2 + y^2} - r$ for a circle—using Boolean algebraic operations, the engine can composite highly complex topographies[2]. Taking the mathematical minimum between two SDFs executes a geometric union, while taking the maximum against a negated SDF executes a geometric subtraction. To simulate organic irregularities and prevent the output from appearing excessively rigid, pseudo-random continuous functions, such as Perlin or Simplex noise, are mathematically injected into the SDF fields. These act as domain warpings, represented by coordinate substitutions such as $f(x + \text{noise}(x, y), y + \text{noise}(x, y))$, effectively introducing controlled, scalable chaos into the otherwise rigid polynomial structures.

Reverse Extraction: Image to Closed-Form Equations

The Reverse Engine faces a significantly more complex, ill-posed inverse problem: mapping an arbitrary discrete pixel matrix $I \in \mathbb{R}^{H \times W \times 3}$ back into a compact, continuous mathematical formula. Multiple algorithmic pathways present varying trade-offs in mathematical accuracy, equation length, and computational cost.

The most traditional approach relies on the 2D Fast Fourier Transform (FFT) and Fourier Epicycles. The 2D FFT decomposes an image into its frequency domain, representing the spatial matrix as an infinite sum of sine and cosine waves. Alternatively, boundary extraction followed by Fourier epicycles parameterizes a one-dimensional contour in the complex plane as $Z(t) = \sum_{k=-N}^{N} c_k e^{ikt}$ [5]. While computationally inexpensive and highly parallelizable, Fourier methods universally yield exceedingly long, dense polynomials. They lack semantic interpretability and consistently fail to capture sharp geometric discontinuities without introducing severe Gibbs ringing, making them unsuitable for generating compact, elegant mathematical formulas.

To achieve higher spatial fidelity, the system can utilize Gaussian Splatting (2DGS), which represents an image as a collection of localized 2D Gaussian primitives. Each primitive is defined by a 2D covariance matrix $\Sigma \in \mathbb{R}^{2 \times 2}$, a position $\mu \in \mathbb{R}^2$, color parameters $c \in \mathbb{R}^3$, and an opacity scalar α [6]. The covariance matrix is parameterized via scaling and rotation matrices as $\Sigma = RSS^T R^T$, allowing the representation of anisotropic shapes[6].

The final image is rendered via differentiable α-compositing, where pixel colors are accumulated front-to-back according to the evaluated Gaussian density at that pixel[7]. Advanced derivatives, such as Quadratic Gaussian Splatting (QGS), replace static primitives

with deformable quadric surfaces, allowing representation of ellipsoids, hyperboloids, and complex cross-sections through a conic matrix formulation $f(x, y, z) = \mathbf{v}^T \mathbf{Q} \mathbf{v} = 0$ [9]. While 2DGS and its variants fit high-fidelity representations very rapidly and inherently provide continuous analytical derivatives, they yield a massive, unstructured parameter matrix. Gaussian Splatting functions essentially as a continuous spatial compression algorithm rather than a true symbolic regression, failing to yield a singular, human-readable closed-form equation.

To bridge the gap between high-fidelity continuous representation and symbolic extraction, PixMath employs a hybrid pipeline combining Implicit Neural Representations (INRs) with Symbolic Regression. Traditional Multi-Layer Perceptrons (MLPs) suffer from a spectral bias, struggling to learn the high-frequency topological details of an image[10]. To overcome this, the architecture utilizes a Sinusoidal Representation Network (SIREN). A SIREN is an MLP that strictly utilizes periodic activation functions, formulated as $\Phi_i(x) = \sin(W_i \Phi_{i-1}(x) + b_i)$ [10]. Because the derivative of a sine wave is simply a phase-shifted sine wave, SIRENs are infinitely differentiable and perfectly suited to capture sharp edges, high-frequency textures, and complex gradients within the pixel matrix[10].

Once the SIREN accurately maps the coordinate space $(x, y) \rightarrow (R, G, B)$, the architecture initiates the second phase: Symbolic Regression. This is achieved through advanced frameworks such as PySR, which utilize evolutionary genetic programming to search the vast combinatorial space of mathematical operators[12]. PySR explores populations of Abstract Syntax Trees (ASTs), executing crossover and mutation operations to find compact algebraic relationships that perfectly mimic the SIREN's internal activations[10]. Alternatively, the system can leverage Kolmogorov-Arnold Networks (KANs). Unlike MLPs which possess fixed activation functions on nodes, KANs place learnable activation functions on the edges, effectively decomposing high-dimensional functions into sums of one-dimensional functions[10]. Because KANs naturally isolate modular structures and relevant features, they can be directly compiled into symbolic formulas using tree conversion algorithms, providing an even more direct route to interpretable symbolic regression without relying entirely on stochastic genetic searches[10].

Quality Metrics and the Pareto Frontier

Evaluating the success of the Reverse Engine requires a delicate balancing act between visual fidelity and mathematical conciseness. A formula that perfectly recreates an image but spans ten thousand pages is functionally useless within the context of PixMath.

Reconstruction accuracy is quantified using traditional pixel-wise metrics like the Peak Signal-to-Noise Ratio (PSNR) alongside structural metrics like the Structural Similarity Index (SSIM). SSIM, and its derivative D-SSIM, specifically measure perceived structural variance, comparing the luminance, contrast, and structural cohesion of the math-generated image against the source[6]. The loss function utilized during the optimization of the implicit representation often combines these metrics, formulated as $\mathcal{L}_{rec} = \lambda \mathcal{L}_2 + (1 - \lambda) \mathcal{L}_{D-SSIM}$, ensuring that both absolute color values and

high-frequency structural details are preserved[6].
Equation complexity is calculated by traversing the generated mathematical equation's Abstract Syntax Tree (AST) and summing the total number of nodes, which includes mathematical operators, variables, and constants[16]. Symbolic regression frameworks optimize a multi-objective loss function to manage this trade-off, balancing the Mean Squared Error (MSE) of the prediction against the complexity score of the tree[12]. The ultimate output of this dual-optimization process is a Pareto frontier: a curve demonstrating the optimal trade-off where any further increase in PSNR demands a logarithmic, prohibitive explosion in equation length[12]. The PixMath engine dynamically analyzes the derivative of this curve to select the equation sitting exactly at the "elbow" of the Pareto frontier, maximizing visual fidelity while ensuring the algebraic formula remains compact and human-readable.

2. Industrial Microservices & Hybrid API Architecture

Because PixMath must handle lightweight, synchronous metadata queries alongside intensely heavy, asynchronous genetic algorithms that run for prolonged periods, the system demands a highly resilient, distributed hybrid architecture.

Hybrid BFF Pattern (Backend-for-Frontend)

The network architecture utilizes a strict Hybrid Backend-for-Frontend (BFF) pattern to orchestrate different network protocols based entirely on the specific payload characteristics and latency requirements of the operation.
The REST API layer is strictly reserved for heavy binary ingestion. When a user uploads a high-resolution target image for the Reverse Engine, REST provides the necessary multipart-form buffering, predictable HTTP chunking, caching mechanisms, and Content Delivery Network (CDN) integration required to handle large payloads efficiently without protocol overhead.
Conversely, GraphQL operates as the primary data-fetching layer for all structured metadata within the platform. Because GraphQL clients define the exact shape and depth of the response, it eliminates the over-fetching and under-fetching dilemmas inherent to REST[17]. This is critical when retrieving heavily nested lists of generated equations, user profiles, or historical render parameters, allowing the frontend client to construct a single, highly specific query rather than orchestrating multiple REST endpoint calls[18].
The most complex network layer involves WebSockets, implemented specifically to handle GraphQL Subscriptions. The genetic programming phase of PySR, or the backpropagation training of a SIREN model, can take anywhere from a few minutes to several hours to converge on an optimal equation[14]. Using the graphql-subscriptions library alongside a Redis PubSub event bus, the system streams live, granular updates directly to the client via stateful WebSockets[17]. As the genetic algorithm traverses the equation space, the WebSocket asynchronously pushes intermediate mathematical discoveries, live PSNR improvements, and serialized AST visualizations. This push-based model completely circumvents the need for the client to execute expensive polling requests, which would unnecessarily drain battery life on mobile devices and flood the API gateway with redundant traffic[21]. Furthermore, utilizing advanced directives like @defer and @stream, the GraphQL server can split requested data

into priority levels, streaming critical metadata first and resolving the heavier AST nodes in subsequent payloads[17].

Event-Driven Compute Pipeline

To decouple the synchronous API gateway from the massive, unpredictable computational footprint of mathematical rendering and SIREN/PySR training, PixMath utilizes RabbitMQ as the core event-driven message broker[22].

The message contract sent to RabbitMQ includes the unique job identifier, a presigned URL pointing to the S3-hosted source image, strict AST complexity boundaries, and the specific hyperparameter configurations required for the genetic algorithm. The configuration of RabbitMQ's consumer mechanisms is paramount to system stability. The prefetch count is arguably the single most critical tuning parameter in this pipeline[23]. Because symbolic regression jobs vary wildly in execution time, leaving the prefetch count at its default setting would result in fast broker exchanges shoving hundreds of messages at a single headless worker[24]. This would lead to rapid memory exhaustion on that node while other workers sit idle. Setting the prefetch count strictly to 1 ensures that a worker node only receives a new computation job upon explicitly acknowledging the completion of the previous one, naturally enforcing backpressure and preventing consumer overload[22].

To prevent data loss in the event of hardware failure, the system mandates Publisher Confirms. This is an asynchronous mechanism that notifies the API gateway only when a message has been correctly stored and persisted to disk within the RabbitMQ queue[22]. If a headless worker crashes due to out-of-memory (OOM) errors while attempting to parse an exponentially complex AST, the unacknowledged message is captured and automatically routed to a Dead-Letter Exchange (DLX)[22]. A secondary, lightweight telemetry service continuously monitors the DLX to trigger automated retries with looser memory constraints, or to alert the end-user that the submitted image has resulted in an intractable mathematical singularity.

Security & AST Sandboxing

Executing user-submitted mathematical expressions within the Forward Engine presents an existential security risk to the platform. In Python, evaluating strings using standard functions like eval() or relying on poorly secured parsing engines can trivially lead to Remote Code Execution (RCE) or directory traversal vulnerabilities if an attacker successfully injects OS-level commands into the mathematical string[28]. Historically, widely used symbolic math libraries like SymPy have contained vulnerabilities within functions such as parse_expr(), allowing malicious payloads to bypass naive string sanitization[28].

To definitively mitigate this threat surface, PixMath employs a defense-in-depth AST sandboxing architecture:

The first layer of defense is rigorous AST Whitelisting. Before any evaluation or transpilation occurs, the user-submitted mathematical string is parsed into an Abstract Syntax Tree (AST)[28]. An AST is a tree-shaped data structure that represents the syntactic structure of the code, completely stripping away surface-level formatting and reducing the input to typed nodes[31]. The system uses an ast.NodeVisitor subclass to traverse this tree programmatically[33]. If the visitor encounters any node type outside a strictly curated whitelist (allowing only safe

mathematical constructs like BinOp, UnaryOp, Num, Name, and specific Call nodes mapped exclusively to safe trigonometric functions), it immediately halts execution and raises a security exception[28].

The second layer addresses resource exhaustion. A malicious user can submit deeply nested expressions, such as 2**(2**(2**2)), or highly repetitive recursive functions, to trigger a stack overflow or trap the CPU in infinite evaluation loops, executing a Denial-of-Service (DoS) attack[28]. The parser enforces a strict structural depth limit on the AST, traversing the nodes to count the maximum vertical depth and the total node volume before any evaluation is permitted[35]. If the AST exceeds these heuristic bounds, the process is aborted.

The third layer is ephemeral containerization. Even with rigorous AST validation, the actual execution of the mathematics occurs within an isolated, ephemeral sandbox, leveraging micro-VM infrastructure such as E2B, Modal, or specialized Docker containers spawned per request[36]. These sandboxes enforce strict Linux cgroups constraints, typically capping the environment at 512MB of RAM and 1 CPU core, with a hard 15-second execution timeout[36]. Furthermore, the network namespace is entirely isolated, restricting all outbound internet access[36]. This ensures that even in the event of a sophisticated zero-day AST escape, the compromised agent cannot exfiltrate environment variables, AWS secrets, or execute DDoS attacks against external targets, thereby fully containing the blast radius[36]. Alternatively, for environments requiring extreme speed and security, parsing and evaluation can be offloaded to robust C++ libraries like ExprTk, which natively provide secure, sandboxed mathematical expression evaluation without the overhead of Python's execution environment[39].

3. Performance Optimization & Hardware Acceleration

Evaluating complex mathematical formulas across millions of distinct spatial coordinates requires specialized processing architectures. The hardware acceleration strategies and serialization formats must be chosen based on the specific compute topology and network constraints of the deployment.

CPU Vectorization vs. GPU Shader Pipelines

Evaluating a continuous mathematical function over a standard 4096×4096 high-resolution image involves over 16.7 million discrete algebraic operations per functional layer.

CPU Vectorization, using standard Python libraries such as NumPy or SciPy for element-wise coordinate evaluation, is fundamentally memory-bound and computationally inefficient at this scale. While Single Instruction, Multiple Data (SIMD) CPU instructions (like AVX-512) can batch mathematical computations to some degree, the Python Global Interpreter Lock (GIL) and the massive memory allocation overheads required to store intermediate matrices severely throttle throughput. Consequently, CPU vectorization within PixMath is restricted solely to parsing the AST and structuring the initial mathematical equations.

The optimal approach leverages massive parallelization through GPU Shader Pipelines. PixMath takes the validated AST generated during the security phase and compiles it directly into GLSL (OpenGL Shading Language). The spatial coordinates (x, y) are inherently parallelizable, as

the evaluation of one pixel's color vector has zero dependency on its neighbors. By utilizing headless EGL contexts via libraries like ModernGL, PixMath can bypass the requirement for a physical display server (X11 or Wayland), allowing it to evaluate the mathematics entirely within GPU VRAM inside containerized environments[42]. This architectural shift reduces render times for multi-million-pixel grids from several seconds on a highly threaded CPU to sub-millisecond latencies on the GPU[42].

Serializing Massive Mathematical Equations

Because the genetic algorithm phase of the Reverse Engine produces millions of massive, highly nested ASTs during PySR optimization, the serialization format's throughput and memory footprint dictate the overall speed of the evolutionary search.

Serialization Format	AST Processing Mechanism	Memory Efficiency & Parsing Speed	Optimal Architectural Use Case
JSON	Full string parsing; requires building dynamic object graphs in memory[43].	Very Low; high serialization overhead and heavy memory bloat[43].	External Web Client and GraphQL API transmission where human readability during debugging is required.
Protocol Buffers	Decodes binary stream into strongly typed memory objects using a schema IDL[44].	Moderate; fast network transfer but still requires memory allocation upon decoding[47].	Internal microservice communication; stable API contracts between the Forward and Reverse engines[48].
FlatBuffers	Zero-copy deserialization; accesses structured data directly via memory offsets and vtables[47].	Exceptional; avoids all memory allocation overhead during parsing[47].	High-frequency storage and caching of massive AST mutation trees directly in memory-constrained worker nodes[47].
LaTeX Strings	Requires complex string regex/parsing to extract discrete	Very Low; extremely high CPU cost for parsing.	Final display rendering exclusively on the

	operators.		client-side browser interface.

While Protocol Buffers are highly efficient and provide a compact interface definition language (IDL), they fundamentally require an unpacking step to create the object graph in memory before the data can be accessed[47]. FlatBuffers, conversely, uses a sophisticated vtable architecture that allows the system to read the AST structure directly from the binary byte buffer without executing a deserialization step[47]. For PixMath, storing the evolutionary tree of equations via FlatBuffers enables instantaneous memory mapping. This completely bypasses the garbage-collection latency and memory allocation spikes inherent to dynamic languages processing large JSON trees, accelerating the evolutionary search algorithm by orders of magnitude[47].

4. Zero-Cost Cloud Stitching & DevOps Constraints

Deploying a mathematically intensive system across serverless edge networks and low-cost cloud tiers (e.g., Cloudflare Workers, Koyeb, Render, Supabase) poses severe operational constraints, primarily revolving around memory caps and cold-start latencies.

Memory, CPU, and Cold-Start Constraints

Serverless edge networks, such as Cloudflare Workers, utilize V8 isolates rather than full container virtualization. While this architecture exhibits near-instantaneous cold starts (under 5 milliseconds), V8 limits execution to JavaScript/WebAssembly environments, severely restricting the use of heavy binary backends like SymPy, PyTorch, or PySR, which rely heavily on native C++ extensions[38]. Conversely, platforms like Render and Koyeb allow for full Dockerized microVM deployments, granting access to Python environments. However, deploying dense scientific computing stacks natively in these environments often results in cold starts exceeding 3 to 10 seconds due to the heavy I/O cost of loading libraries into memory[38].

The most pressing constraint across free-tier headless workers is the hard memory limit, typically capped at 512 MB of RAM. Attempting to run a continuous optimization loop for a SIREN neural network alongside a concurrent symbolic regression algorithm (PySR) on a single node will instantly trigger an Out-Of-Memory (OOM) kill. The evolutionary generation, mutation, and evaluation of thousands of complex ASTs causes rapid heap fragmentation and massive memory spikes that standard Python garbage collection cannot mitigate quickly enough.

Architectural Workarounds for 512 MB Node Constraints

To successfully execute PixMath's Reverse Engine within these draconian limitations, the system utilizes aggressive, distributed memory management strategies:

The primary workaround involves a Distributed MapReduce architecture for the evolutionary algorithms. Instead of running the entire PySR genetic population on a single monolithic node, PixMath shards the population array across multiple stateless serverless workers. Each worker operates its localized genetic mutation batch well within its 512 MB limit, scores the equations, and pushes the surviving "fittest" ASTs to a centralized Redis cache or a Supabase pgvector database[36]. A lightweight coordinator node then aggregates these results, executes the

crossover functions, and triggers the next generation of workers.
Furthermore, leveraging the aforementioned FlatBuffers serialization enables streaming AST parsing with zero-copy I/O. This allows the worker nodes to read historical equation states directly from disk or memory-mapped files without ever allocating massive object trees into the active RAM, bypassing the primary cause of OOM failures[47].
Finally, the architecture enforces aggressive Garbage Collection and Lazy Loading. Heavy scientific modules like sympy, scipy, and torch consume over 200MB of RAM simply upon import. PixMath relies on dynamic module loading (importlib) to bring these libraries into memory only at the exact microsecond they are required for execution[37]. Once a SIREN training loop completes its forward and backward passes, the GPU/CPU tensors are explicitly deallocated, variables are unreferenced, and synchronous garbage collection is forcefully invoked before the PySR symbolic regression phase is permitted to begin.

5. Competitive Landscape & Open-Source Gap Analysis

PixMath occupies a highly specialized intersection between analytical mathematics, generative artificial intelligence, and hardware vectorization, effectively bridging gaps left by existing industrial and mathematical standards.

Platform	Forward Rendering Engine	Reverse Extraction Capabilities	Symbolic Export Format	Architectural Bottlenecks & Limitations	The PixMath Advantage
Wolfram Mathematica	Excellent (CPU-bound analytical rendering)	None / Extremely limited symbolic extraction from pixels	Analytical Mathematics	Proprietary, cost-prohibitive, primarily CPU-bound, and requires highly specialized Wolfram language syntax.	Fully open-source AST representation, massively parallel GPU-accelerated EGL execution, and true bidirectional capabilities.

Shadertoy	Excellent (Native GPU/GLSL acceleration)	None	GLSL Code	Highly manual; relies purely on forward code writing; completely lacks a symbolic or algebraic math engine.	Abstracts complex GLSL implementation via safe ASTs; auto-generates shader code seamlessly from the AI Reverse Engine.
Desmos	Good (2D graphing)	None	Basic Algebraic Math	Cannot handle complex periodic composites, procedural noise, SDFs, or millions of pixels efficiently.	Boundless dimensional rendering; capable of mapping complex real-world objects and textures via periodic topologies.
Potrace	None	Yes (Strictly Bitmap to SVG)	Vector Paths (Bezier Curves)	Only traces high-contrast boundaries into hard-coded Bezier curves; lacks any understanding of deep underlying mathematical forms[5].	Derives fundamental periodic, trigonometric, and algebraic formulas, providing deep semantic representation rather than mere surface vectors.

While tools like Potrace accurately convert discrete pixels to scalable vectors, they only output deterministic spatial control points. PixMath identifies the underlying *generative algorithm*—such as the exact nested trigonometric function that dictated the shape and texture of a curve—creating a heavily compressed, semantic representation of the image rather than a superficial tracing.

6. Skills Matrix, Critical Questions & Phased Roadmap

Prerequisite Skills Matrix

To successfully build, scale, and maintain the PixMath architecture, the engineering organization must possess a deeply intersecting matrix of expertise:

- **Applied Mathematics:** High proficiency in non-Euclidean geometry, Fourier analysis, parametric spatial representations, and the topological application of Signed Distance Functions.
- **AI / Machine Learning:** Extensive experience with Implicit Neural Representations (INRs), Multi-Layer Perceptrons (MLPs), Kolmogorov-Arnold Networks (KANs), and the tuning of genetic programming/evolutionary algorithms.
- **Distributed Systems Architecture:** Mastery of RabbitMQ message broker topologies, AMQP protocols, stateful WebSockets, and GraphQL subscription lifecycle management.
- **Systems Programming & Security:** Low-level memory management (FlatBuffers, C++ integration), Abstract Syntax Tree (AST) parsing algorithms, and Linux virtualization/cgroups sandboxing.

Top 10 Hardest Engineering Bottlenecks

1. **SIREN Spectral Bias & Overfitting:** Finely tuning the periodic activation frequencies to prevent the neural network from memorizing pixel noise while still perfectly preserving high-frequency structural edges.
2. **PySR Convergence Latency:** Symbolic regression remains an NP-hard combinatorial problem; preventing the genetic algorithm from stalling in local minima within restrictive serverless timeout windows requires advanced heuristic tuning.
3. **AST Memory Bloat:** Managing the exponential growth of AST nodes during PySR crossover mutations without triggering out-of-memory kills on edge nodes.
4. **Sandbox Breakouts:** Definitively ensuring that the SymPy parsing engine cannot be bypassed via obfuscated variable injection, recursive __getattr__ chaining, or memory overflow exploits.
5. **WebGL/EGL Headless Contexts:** Ensuring the reliable spinning up of GPU contexts inside headless Docker containers without relying on traditional display servers (X11/Wayland), which are absent in cloud environments.
6. **WebSocket State Management:** Handling client disconnects and reconnections seamlessly without losing the real-time stream of the PySR Pareto frontier evolution.
7. **RabbitMQ Backpressure:** Tuning prefetch counts and Dead-Letter Exchanges across a cluster of heterogenous workers where job computation times range dynamically from 2 seconds to 45 minutes.
8. **Equation Simplicity vs. Fidelity:** Algorithmically determining the exact mathematical

cut-off point on the Pareto curve that dictates subjective "human readability" over raw PSNR maximization.

9. **Cold Start Latencies:** Mitigating the massive payload size of libraries like PyTorch, SymPy, and PySR in serverless and microVM deployments through lazy loading and environment caching.
10. **GLSL Translation Integrity:** Ensuring that highly specific mathematical edge cases (e.g., division by zero, complex logarithms, negative roots) within the Python AST translate cleanly and safely into parallelized GPU shaders without crashing the rendering pipeline.

Step-by-Step MVP-to-Production Roadmap

Phase 1: Mathematical Foundations & Forward MVP (Weeks 1-4)

- Define the core ast.NodeVisitor sandbox to securely parse, validate, and restrict text strings into highly constrained mathematical trees.
- Implement the baseline CPU-bound evaluation engine using NumPy/SciPy to establish unit testing for mathematical correctness.
- Design and test the translation layer that dynamically compiles the sanitized Python AST into highly parallelizable GLSL code.
- Deploy a headless ModernGL/EGL container capable of executing the generated GLSL on GPU hardware, mapping the x, y coordinate space directly to RGB outputs.

Phase 2: The Reverse Engine (Weeks 5-9)

- Integrate the PyTorch framework and construct the foundational SIREN and KAN architectures.
- Implement the image-to-SIREN training loop, continuously validating output fidelity against PSNR and D-SSIM loss functions.
- Connect the PySR symbolic regression framework to distill the optimized neural network weights into closed-form ASTs.
- Establish the Pareto frontier mathematical selection logic to extract the most optimal, human-readable equation from the evolutionary population.

Phase 3: Event-Driven Infrastructure & BFF (Weeks 10-14)

- Deploy the RabbitMQ cluster and configure the strict prefetch count limitations, publisher confirms, and DLX routing policies.
- Develop the GraphQL API for historical metadata fetching and the REST layer for large binary image ingestion.
- Implement the WebSocket and GraphQL Subscription layers, utilizing Redis PubSub, to stream PySR's evolutionary progress back to the client interface in real time.
- Transition all internal data serialization architectures from JSON to FlatBuffers for high-speed, zero-copy worker state caching.

Phase 4: Optimization, Security, and Production (Weeks 15-18)

- Implement Docker/E2B ephemeral microVMs to enforce strict 512 MB memory constraints and CPU limits on all execution nodes.
- Configure the lazy loading sequences and distributed MapReduce strategies to manage the PySR evolutionary populations across multiple workers.
- Execute rigorous Red Team penetration testing specifically targeting the AST parser and

sandbox isolation boundaries.
- Finalize comprehensive load testing, set up DLX monitoring telemetry, and execute the public deployment.

Through this exhaustive architectural layout, PixMath seamlessly binds the abstract logic of algebraic geometry with the visual intuition of computer graphics, secured, optimized, and scaled via modern, event-driven industrial topologies.

Works cited

1. Hamid Naderi Yeganeh - Wikipedia, https://en.wikipedia.org/wiki/Hamid_Naderi_Yeganeh
2. Artist Hamid Yeganeh draws these using only mathematical equations, https://www.reddit.com/r/Damnthatsinteresting/comments/1ldvs9u/artist_hamid_yeganeh_draws_these_using_only/
3. How to Draw with Math - Palestinian Genocide Weblog, https://azadpalestine.wordpress.com/2017/01/12/how-to-draw-with-math/
4. ICM 2026 - Imaginary.org, https://www.imaginary.org/gallery/icm-2026
5. Visualization — list of Rust libraries/crates // Lib.rs, https://lib.rs/visualization
6. Instant GaussianImage: A Generalizable and Self-Adaptive Image, https://arxiv.org/html/2506.23479v1
7. Fast Multi-View Reconstruction with Generalizable 2D Gaussian, https://arxiv.org/html/2505.02175v1
8. Introducing Unbiased Depth into 2D Gaussian Splatting for High, https://arxiv.org/html/2503.06587v1
9. Quadratic Gaussian Splatting: High Quality Surface Reconstruction, https://arxiv.org/html/2411.16392
10. (PDF) Kolmogorov-Arnold Networks Meet Science - ResearchGate, https://www.researchgate.net/publication/398797888_Kolmogorov-Arnold_Networks_Meet_Science
11. KAN: Kolmogorov–Arnold Networks - Хабр, https://habr.com/ru/articles/856776/
12. KAN: Kolmogorov–Arnold Networks - arXiv, https://arxiv.org/html/2404.19756v1
13. Singularity Formation: Synergy in Theoretical, Numerical and ... - arXiv, https://arxiv.org/html/2604.16842v1
14. 26TH INTERNATIONAL CONFERENCE ON COMPUTING IN HIGH, https://indico.jlab.org/event/459/timetable/?print=1&view=standard
15. Using Gaussian Splats to Create High-Fidelity Facial Geometry and, https://arxiv.org/html/2512.16397v1
16. APPL: A Prompt Programming Language for Harmonious Integration, https://aclanthology.org/2025.acl-long.63.pdf
17. Types of Realtime GraphQL Subscriptions - Hasura, https://hasura.io/blog/types-of-realtime-graphql-subscriptions
18. Subscriptions - GraphQL, https://graphql.org/learn/subscriptions/
19. Subscriptions | GraphQL Modules, https://the-guild.dev/graphql/modules/docs/advanced/subscriptions
20. Tutorial: GraphQL Subscriptions on the Server,

https://www.apollographql.com/blog/tutorial-graphql-subscriptions
21. GraphQL Subscriptions: Real-Time Data Fetching | by Sohail x Codes, https://medium.com/@sohail_saifii/graphql-subscriptions-real-time-data-fetching-51f74f341ef5
22. RabbitMQ: A Complete Guide to Message Broker, Performance, and, https://medium.com/@erickzanetti/rabbitmq-a-complete-guide-to-message-broker-performance-and-reliability-3999ee776d85
23. RabbitMQ Prefetch Count: Optimal Configuration | AceMQ, https://acemq.com/blogs/rabbitmq-prefetch-count/
24. RabbitMQ Data Pipelines: Scale ETL without adding compute, https://www.thehammadtariq.com/blog/scaling-data-pipelines-with-rabbitmq
25. Quorum Queues and Flow Control - The Concepts - RabbitMQ, https://www.rabbitmq.com/blog/2020/05/04/quorum-queues-and-flow-control-the-concepts
26. Consumer Acknowledgements and Publisher Confirms - RabbitMQ, https://www.rabbitmq.com/docs/confirms
27. How to Implement Publisher Confirms in RabbitMQ - OneUptime, https://oneuptime.com/blog/post/2026-01-26-rabbitmq-publisher-confirms/view
28. Python eval: is it still dangerous if I disable builtins and attribute, https://stackoverflow.com/questions/35804961/python-eval-is-it-still-dangerous-if-i-disable-builtins-and-attribute-access
29. Visual Studio Code Settings Sync Gist · GitHub, https://gist.github.com/wanghmalan/2917bd408221ffe339be7c0e9c00ca5e
30. Vulnerability Summary for the Week of August 24, 2026 | CISA, https://www.cisa.gov/news-events/bulletins/sb26-243
31. Abstract Syntax Tree (AST) Deep Dive: From Theory to Practical, https://dev.to/min_yi_e5fbf986e24f1c42df/abstract-syntax-tree-ast-deep-dive-from-theory-to-practical-compiler-implementation-4jpo
32. What Is an Abstract Syntax Tree (AST)? How ASTs Power Modern, https://macroscope.com/content/what-is-abstract-syntax-tree
33. Evaluating a mathematical expression in a string - Stack Overflow, https://stackoverflow.com/questions/2371436/evaluating-a-mathematical-expression-in-a-string
34. FoundriesFactory Release 86 (v. 1554) - Foundries.io, https://www.foundries.io/products/releases/86/
35. Limiting the depth of ASTs to improve reliability and security - Ideas, https://discuss.python.org/t/limiting-the-depth-of-asts-to-improve-reliability-and-security/18787
36. What Are AI Harnesses? Reliable Agent Architecture - Promptnote, https://promptnote.app/blogs/what-are-ai-harnesses/
37. Code Execution | Gemini Enterprise Agent Platform, https://docs.cloud.google.com/gemini-enterprise-agent-platform/scale/sandbox/code-execution-overview
38. Why Do AI Models and Autonomous Agents Overwhelmingly Favor, https://blog.margrop.net/en/post/why-ai-prefers-python-and-nodejs-deep-dive/

39. cexprtk · PyPI, https://pypi.org/project/cexprtk/
40. C++ Mathematical Expression Library (ExprTk) - By Arash Partow, https://www.partow.net/programming/exprtk/
41. exprtk/0.0.3 - Conan 2.0: C and C++ Open Source Package Manager, https://conan.io/center/recipes/exprtk
42. How to run moderngl in Colab? - opengl - Stack Overflow, https://stackoverflow.com/questions/60900067/how-to-run-moderngl-in-colab
43. fast JSON data access using JIT-based speculative optimizations, https://www.researchgate.net/publication/319597745_FADjs_fast_JSON_data_access_using_JIT-based_speculative_optimizations
44. Protocol Buffer- Protobuf in System Design - GeeksforGeeks, https://www.geeksforgeeks.org/system-design/protocol-buffer-protobuf-in-system-design/
45. Basics - Protobuf Explained, https://protobuf.com/basics/
46. Protobuffers Are Wrong (2018) - Hacker News, https://news.ycombinator.com/item?id=45139656
47. ACES - ACCESS Allocations, https://allocations.access-ci.org/resources/aces.tamu.access-ci.org?QXS49lKOMpZO=AfgSw0v8
48. Evaluation of Efficiency and Performance of Serialization Formats for, https://www.researchgate.net/publication/387095577_Evaluation_of_Efficiency_and_Performance_of_Serialization_Formats_for_Distributed_Systems
49. Embedded Remote Procedure Call Code Generation Tool - IS MUNI, https://is.muni.cz/th/mwjt8/diploma_thesis_misik.pdf